

Programming Languages

PRINCIPLES AND PRACTICE

KENNETH C. LOUDEN & KENNETH A. LAMBERT

THIRD EDITION

Programming Languages

Principles and Practice

Third Edition

Kenneth C. Louden

San Jose State University

Kenneth A. Lambert

Washington and Lee University



Australia • Brazil • Japan • Korea • Mexico • Singapore • Spain • United Kingdom • United States

This is an electronic version of the print textbook. Due to electronic rights restrictions, some third party content may be suppressed. Editorial review has deemed that any suppressed content does not materially affect the overall learning experience. The publisher reserves the right to remove content from this title at any time if subsequent rights restrictions require it. For valuable information on pricing, previous editions, changes to current editions, and alternate formats, please visit www.cengage.com/highered to search by ISBN#, author, title, or keyword for materials in your areas of interest.



**Programming Languages: Principles
and Practice, Third Edition**
Kenneth C. Louden and Kenneth A. Lambert

Executive Editor: Marie Lee
Acquisitions Editor: Brandi Shailer
Senior Product Manager: Alyssa Pratt
Development Editor: Ann Shaffer
Editorial Assistant: Jacqueline Lacaire
Associate Marketing Manager:
Shanna Shelton
Content Project Manager: Jennifer Feltri
Art Director: Faith Brosnan
Print Buyer: Julio Esperas
Cover Designer: Saizon Design
Cover Photo: © Ocean/Corbis
Compositor: Integra
Copyeditor: Foxxe Editorial
Proofreader: Christine Clark
Indexer: Sharon Hilgenberg

ALL RIGHTS RESERVED. No part of this work covered by the copyright herein may be reproduced, transmitted, stored or used in any form or by any means graphic, electronic, or mechanical, including but not limited to photocopying, recording, scanning, digitizing, taping, Web distribution, information networks, or information storage and retrieval systems, except as permitted under Section 107 or 108 of the 1976 United States Copyright Act, without the prior written permission of the publisher.

For product information and technology assistance, contact us at
Cengage Learning Customer & Sales Support, 1-800-354-9706

For permission to use material from this text or product, submit all
requests online at www.cengage.com/permissions
Further permissions questions can be emailed to
permissionrequest@cengage.com

Library of Congress Control Number: 2010939435

ISBN-13: 978-1-111-52941-3

ISBN-10: 1-111-52941-8

Course Technology
20 Channel Center Street
Boston, MA 02210
USA

Course Technology, a part of Cengage Learning, reserves the right to revise this publication and make changes from time to time in its content without notice.

The programs in this book are for instructional purposes only.

They have been tested with care, but are not guaranteed for any particular intent beyond educational purposes. The author and the publisher do not offer any warranties or representations, nor do they accept any liabilities with respect to the programs.

Cengage Learning is a leading provider of customized learning solutions with office locations around the globe, including Singapore, the United Kingdom, Australia, Mexico, Brazil, and Japan. Locate your local office at:
www.cengage.com/global

Cengage Learning products are represented in Canada by
Nelson Education, Ltd.

To learn more about Course Technology, visit
www.cengage.com/coursestechnology

Purchase any of our products at your local college store or at our preferred online store www.cengagebrain.com

Printed in the United States of America
1 2 3 4 5 6 7 17 16 15 14 13 12 11



Preface v

Chapter 1

Introduction

- 1.1 The Origins of Programming Languages3
- 1.2 Abstractions in Programming Languages8
- 1.3 Computational Paradigms15
- 1.4 Language Definition16
- 1.5 Language Translation18
- 1.6 The Future of Programming Languages19

Chapter 2

Language Design Criteria

- 2.1 Historical Overview27
- 2.2 Efficiency28
- 2.3 Regularity30
- 2.4 Security33
- 2.5 Extensibility34
- 2.6 C++: An Object-Oriented Extension of C . . .35
- 2.7 Python: A General-Purpose Scripting
Language38

Chapter 3

Functional Programming

- 3.1 Programs as Functions47
- 3.2 Scheme: A Dialect of Lisp50
- 3.3 ML: Functional Programming with
Static Typing65
- 3.4 Delayed Evaluation77
- 3.5 Haskell—A Fully Curried Lazy Language
with Overloading81
- 3.6 The Mathematics of Functional
Programming: Lambda Calculus90

Chapter 4

Logic Programming

- 4.1 Logic and Logic Programs105
- 4.2 Horn Clauses109
- 4.3 Resolution and Unification111
- 4.4 The Language Prolog115
- 4.5 Problems with Logic Programming126
- 4.6 Curry: A Functional Logic Language131

Chapter 5

Object-Oriented Programming

- 5.1 Software Reuse and Independence143
- 5.2 Smalltalk144
- 5.3 Java162
- 5.4 C++181
- 5.5 Design Issues in Object-Oriented
Languages191
- 5.6 Implementation Issues in Object-Oriented
Languages195

Chapter 6

Syntax

- 6.1 Lexical Structure of Programming
Languages204
- 6.2 Context-Free Grammars and BNFs208
- 6.3 Parse Trees and Abstract
Syntax Trees213
- 6.4 Ambiguity, Associativity, and
Precedence216
- 6.5 EBNFs and Syntax Diagrams220
- 6.6 Parsing Techniques and Tools224
- 6.7 Lexics vs. Syntax vs. Semantics235
- 6.8 Case Study: Building a Syntax Analyzer
for TinyAda237

Chapter 7

Basic Semantics

- 7.1 Attributes, Binding, and Semantic
Functions257
- 7.2 Declarations, Blocks, and Scope260
- 7.3 The Symbol Table269
- 7.4 Name Resolution and Overloading282
- 7.5 Allocation, Lifetimes, and the
Environment289
- 7.6 Variables and Constants297
- 7.7 Aliases, Dangling References, and
Garbage303
- 7.8 Case Study: Initial Static Semantic
Analysis of TinyAda309

Chapter 8

Data Types

8.1 Data Types and Type Information	328
8.2 Simple Types	332
8.3 Type Constructors	335
8.4 Type Nomenclature in Sample Languages	349
8.5 Type Equivalence	352
8.6 Type Checking	359
8.7 Type Conversion	364
8.8 Polymorphic Type Checking	367
8.9 Explicit Polymorphism	376
8.10 Case Study: Type Checking in TinyAda	382

Chapter 9

Control I—Expressions and Statements

9.1 Expressions	403
9.2 Conditional Statements and Guards	410
9.3 Loops and Variations on WHILE	417
9.4 The GOTO Controversy and Loop Exits	420
9.5 Exception Handling	423
9.6 Case Study: Computing the Values of Static Expressions in TinyAda	432

Chapter 10

Control II—Procedures and Environments

10.1 Procedure Definition and Activation	445
10.2 Procedure Semantics	447
10.3 Parameter-Passing Mechanisms	451
10.4 Procedure Environments, Activations, and Allocation	459
10.5 Dynamic Memory Management	473
10.6 Exception Handling and Environments	477
10.7 Case Study: Processing Parameter Modes in TinyAda	479

Chapter 11

Abstract Data Types and Modules

11.1 The Algebraic Specification of Abstract Data Types	494
11.2 Abstract Data Type Mechanisms and Modules	498
11.3 Separate Compilation in C, C++ Namespaces, and Java Packages	502
11.4 Ada Packages	509
11.5 Modules in ML	515
11.6 Modules in Earlier Languages	519
11.7 Problems with Abstract Data Type Mechanisms	524
11.8 The Mathematics of Abstract Data Types	532

Chapter 12

Formal Semantics

12.1 A Sample Small Language	543
12.2 Operational Semantics	547
12.3 Denotational Semantics	556
12.4 Axiomatic Semantics	565
12.5 Proofs of Program Correctness	571

Chapter 13

Parallel Programming

13.1 Introduction to Parallel Processing	583
13.2 Parallel Processing and Programming Languages	587
13.3 Threads	595
13.4 Semaphores	604
13.5 Monitors	608
13.6 Message Passing	615
13.7 Parallelism in Non-Imperative Languages	622

This book is an introduction to the broad field of programming languages. It combines a general presentation of principles with considerable detail about many modern languages. Unlike many introductory texts, it contains significant material on implementation issues, the theoretical foundations of programming languages, and a large number of exercises. All of these features make this text a useful bridge to compiler courses and to the theoretical study of programming languages. However, it is a text specifically designed for an advanced undergraduate programming languages survey course that covers most of the programming languages requirements specified in the 2001 ACM/IEEE-CS Joint Curriculum Task Force Report, and the CS8 course of the 1978 ACM Curriculum.

Our goals in preparing this new edition are to bring the language-specific material in line with the changes in the popularity and use of programming languages since the publication of the second edition in 2003, to improve and expand the coverage in certain areas, and to improve the presentation and usefulness of the examples and exercises, while retaining as much of the original text and organization as possible. We are also mindful of the findings and recommendations of the ACM SIGPLAN Programming Language Curriculum Workshop [2008], which reaffirm the centrality of the study of programming languages in the computer science curriculum. We believe that the new edition of our book will help students to achieve the objectives and outcomes described in the report, which was compiled by the leading teachers in our field.

To complete this book, students do not have to know any one particular language. However, experience with at least one language is necessary. A certain degree of computational sophistication, such as that provided by a course in data structures (CS2) and a discrete mathematics course, is also expected. A course in computer organization, which provides some coverage of assembly language programming and virtual machines, would be useful but is not essential. Major languages used in this edition include C, C++, Smalltalk, Java, Ada, ML, Haskell, Scheme, and Prolog; many other languages are discussed more briefly.

Overview and Organization

In most cases, each chapter largely is independent of the others without artificially restricting the material in each. Cross references in the text allow the student or instructor to fill in any gaps that might arise even if a particular chapter or section is skipped.

Chapter 1 surveys the concepts studied in later chapters, provides an overview of the history of programming languages, and introduces the idea of abstraction and the concept of different language paradigms.

Chapter 2 provides an overview of language design criteria. Chapter 2 could serve well as a culminating chapter for the book, but we find it arouses interest in later topics when covered here.

Chapters 3, 4, and 5 concretely address three major language paradigms, beginning with the function-oriented paradigm in Chapter 3. Scheme, ML, and Haskell are covered in some detail. This chapter also introduces the lambda calculus. Chapter 4, on logic programming, offers an extended section on Prolog, and devotes another section to the functional logic language Curry. Chapter 5 deals with the object-oriented paradigm. We use Smalltalk to introduce the concepts in this chapter. Individual sections also feature Java and C++.

Chapter 6 treats syntax in some detail, including the use of BNF, EBNF, and syntax diagrams. A brief section treats recursive definitions (like BNF) as set equations to be solved, a technique that recurs periodically throughout the text. One section is devoted to recursive-descent parsing and the use of parsing tools. The final section of this chapter begins a multi-chapter case study that develops a parser for a small language similar to Ada.

Chapters 7, 8, 9, and 10 cover the central semantic issues of programming languages: declaration, allocation, evaluation; the symbol table and runtime environment as semantic functions; data types and type checking; procedure activation and parameter passing; and exceptions and exception handling.

Chapter 11 gives an overview of modules and abstract data types, including language mechanisms for equational, or algebraic, specification.

Chapter 12 introduces the three principal methods of formal semantics: operational, denotational, and axiomatic. This is somewhat unique among introductory texts in that it gives enough detail to provide a real flavor for the methods.

Chapter 13 discusses the major ways parallelism has been introduced into programming languages: coroutines, threads, semaphores, monitors, and message passing, with examples primarily from Java and Ada. Its final section surveys recent efforts to introduce parallelism into LISP and Prolog, and the use of message passing to support parallel programming in the functional language Erlang.

Use as a Text

Like any programming languages text, this one covers a great deal of material. It should be possible to cover all of it in a two-semester or two-quarter sequence. Alternatively, there are two other, very different ways of delivering this material. They could loosely be called the “principles” approach and the “paradigm” approach. Two suggested organizations of these approaches in a semester-long course are as follows:

The principles approach: Chapters 1, 2, 3, 6, 7, 8, 9, and 10.

The paradigm approach: Chapters 1, 2, 3, 4, 5, 6, 7, 8, and 13. If there is extra time, selected topics from the remaining chapters.

Summary of Changes between the Second and Third Editions

The most obvious change from the second edition is the shifting of the three chapters on non-imperative programming languages to a much earlier position in the book (from Chapters 10-12 to Chapters 3-5, with the chapter on object-oriented programming now coming after those on functional and logic

programming). As a consequence, the chapters on syntax and semantics now appear a bit later (Chapters 6-10 instead of 4-8). There are several reasons for this rearrangement:

1. By being exposed early to programming languages and paradigms that they may not have seen, students will gain perspective on the language and paradigm that they already have used, and thus become aware of their power and their limitations.
2. Students will have an opportunity to write programs in one or more new languages much earlier in the course, thus giving them an opportunity to become proficient in alternative styles of programming.
3. The practical experience with some interesting and powerful new languages early in the course will build students' motivation for examining the more theoretical topics explored later, in the chapters on syntax and semantics.

Additional significant changes are as follows:

- The material on the history of programming languages in Chapter 2 has been condensed and moved to Chapter 1, thus shortening the book by one chapter. A brief discussion of machine language and assembly language has also been added to this chapter.
- A case study on the design of Python, a popular general-purpose scripting language, now follows the case study on C++ in Chapter 2. The two case studies illustrate the tradeoffs that occur when designing new languages.
- The chapter on object-oriented programming is now the last of the three chapters on programming paradigms instead of the first one. The order of these chapters now reflects the increasing complexity of the underlying models of computation of each programming paradigm (functions, logic, objects).
- The section on Scheme in the chapter on functional programming has been substantially rewritten and expanded.
- Object-oriented programming in Chapter 5 is now introduced with Smalltalk rather than Java. This new order of presentation will allow students to learn how a language was cleanly built around object-oriented concepts, before they see the tradeoffs and compromises that designers had to make in designing Java and C++.
- The section on Java in the chapter on object-oriented programming has been updated to include a discussion of interfaces, generic collections, and iterators.
- The section on logical constraint languages in the chapter on logic programming has been replaced with a discussion of the functional logic language Curry.
- Beginning in Chapter 6, on syntax, and extending through the Chapters 7-10, on semantics, new end-of-chapter sections present a case study of a parser for a small language that resembles Ada. The design of this software is presented

incrementally, starting with a raw syntax analyzer and adding features to handle static semantic analysis, such as scope analysis and type checking. This new case study will give students extra practical experience with the concepts they learn in each of these chapters.

- A brief discussion of Erlang, a functional programming language that uses message passing to support concurrent processing, has been added to Chapter 13 on parallel programming.

Instructor and Student Resources

The following supplemental materials are available when this book is used in a classroom setting. All of the resources available with this book are provided to the instructor on a single CD-ROM, and most are also available at login.cengage.com.

- **Electronic Instructor's Manual.** The Instructor's Manual that accompanies this textbook includes additional instructional material to assist in class preparation, including items such as Sample Syllabi, Chapter Outlines, Technical Notes, Lecture Notes, Quick Quizzes, Teaching Tips, Discussion Topics, and Sample Midterm and Final Projects.
- **ExamView®.** This textbook is accompanied by ExamView, a powerful testing software package that allows instructors to create and administer printed, computer (LAN-based), and Internet exams. ExamView includes hundreds of questions that correspond to the topics covered in this text, enabling students to generate detailed study guides that include page references for further review. The computer-based and Internet testing components allow students to take exams at their computers, and also save the instructor time by grading each exam automatically.
- **PowerPoint Presentations.** This book comes with Microsoft PowerPoint slides for each chapter. These are included as a teaching aid for classroom presentation and can be made available to students on the network for chapter review or printed for classroom distribution. Instructors can add their own slides for additional topics they introduce to the class.
- **Solution Files.** Selected answers for many of the exercises at the end of each chapter may be found on the Instructor Resources CD-ROM, or at login.cengage.com. Many are programming exercises (most rather short) focusing on languages discussed in the text. Conceptual exercises range from short-answer questions that test understanding of the material to longer, essay-style exercises and challenging "thought" questions. A few moments' reflection should give the reader adequate insight into the potential difficulty of a particular exercise. Further knowledge can be gained by reading the on-line answers, which are treated as an extension of the text and sometimes provide additional information beyond that required to solve the problem. Occasionally

the answer to an exercise on a particular language requires the reader to consult a language reference manual or have knowledge of the language not specifically covered in the text. Complete program examples are available through www.cengage.com. The author's Web site, at home.wlu.edu/~lambertk, also contains links to free, downloadable translators for all the major languages discussed in the book, many of which were used to test the examples.

- **Distance Learning.** Course Technology is proud to present online test banks in WebCT and Blackboard, to provide the most complete and dynamic learning experience possible. Instructors are encouraged to make the most of the course, both online and offline. For more information on how to access your online test bank, contact your local Course Technology sales representative.

Acknowledgments

Ken Loudon would like to thank all those persons too numerous to mention who, over the years, have emailed him with comments, corrections, and suggestions. He remains grateful to the many students in his CS 152 sections at San Jose State University for their direct and indirect contributions to the first and second editions, and to his colleagues at San Jose State, Michael Beeson, Cay Horstmann, and Vinh Phat, who read and commented on individual chapters in the first edition.

Ken Lambert would like to thank his colleagues at Washington and Lee University, Tom Whaley, Simon Levy, and Joshua Stough, and his students in Computer Science 312, for many productive discussions of programming language issues and problems. He also greatly appreciates the work of the reviewers of this edition: Karina Assiter, Wentworth Institute of Technology; Dave Musicant, Carleton College; Amar Raheja, California State Polytechnic University, Pomona; Christino Tamon, Clarkson University.

He would be grateful to receive reports of errors and any other comments from readers at lambertk@wlu.edu.

Ken Lambert offers special thanks to all the people at Course Technology who helped make the third edition a reality, including Brandi Shailer, Acquisitions Editor; Alyssa Pratt, Senior Product Manager; Ann Shaffer, Development Editor; Jennifer Feltri, Content Project Manager. Also, thanks to Amrin Sahay, of Integra Software Services, for overseeing the process of transforming the manuscript into the printed book. Many thanks to Chris Scriver, MQA Project Leader, for ably overseeing the quality assurance testing, as well as to Teresa Storch and Serge Palladino, quality assurance testers, for their many helpful suggestions and corrections.

Finally, both authors would like to thank their wives and children for their love and support.

Notes and References

The report of the ACM SIGPLAN Programming Language Curriculum Workshop appears in *SIGPLAN Notices*, Volume 43, Number 11, November, 2008.